

CS5224 Cloud Computing

Lab 2: Cloud Services

Submission Deadline: 26th October 2026, 18:00 (*submit through Canvas*)

Objective

The purpose of this lab is to evaluate your basic understanding of cloud services, their appropriate usage and to implement a data analytics pipeline. For the purpose of this lab, you can use any cloud provider. The available free tier should be more than sufficient.

Answer the questions (**include necessary screenshots of code and result**) in a PDF Document (`YourStudentID.pdf`, e.g., `A0123456A.pdf`) and submit to Canvas. A penalty of **1** mark would be applied to any submission not following the naming convention. **Do not** worry about the auto numbering Canvas adds to any resubmissions.

Part 1: Cloud Services (4 marks)

- a) Design a cloud strategy for a healthcare analytics SaaS used by multiple hospitals. Patient data must be tightly governed, demand can spike during outbreaks, and the system must still scale nationally. What deployment model(s) and scaling architecture(s) would you choose, and why?
- b) A bank wants to move away from its legacy application and modernise it with strict compliance and some sensitive data. Which cloud service and cloud deployment model would you use and why?

Part 2: Data Analyse It! (6 marks)

To display the final results, you will use a full-stack web application with a front-end and a back-end, the latter including the component used for the data analysis. In this part, you will develop a web application that internally uses a data analysis pipeline (MapReduce, Spark, etc.) to compute the total number of followers and followees for each user in a Twitter dataset. The dataset is provided to you in the file `twitter_combined.txt` (taken from <https://snap.stanford.edu/data/egonets-Twitter.html> and updated). The format of the file is as follows.

- Each line of the file contains two user IDs.
- Line 1 of the file contains `214328887 34428380`. This implies that user `214328887` follows user `34428380`.
- The data is presented as is and not cleaned up, so you have to ensure you take care of any duplicates, etc.

1. Front-End (2 marks)

The front-end fetches data from the back-end by calling backend APIs and presents user statistics in the following format:

```
UserID1 has Number_of_followers
UserID1 has Number_of_followees
UserID2 has Number_of_followers
UserID2 has Number_of_followees
UserID3 has Number_of_followers
UserID3 has Number_of_followees
...
UserIDN has Number_of_followers
UserIDN has Number_of_followees
```

where a “followee” of `UserIDN` represents a user that is followed by `UserIDN`.

In your submission, you should show the results for user 214328887 and user 107830991.

You need to provide a short description (max 50 words) of the set-up of your front-end. For example, about the technologies used and how you connect to the backend.

2. Back-End (2 marks)

The back-end acts as the core hub, managing the provided dataset (e.g., from AWS S3, Google Cloud Storage, Azure Blob Storage, etc.) and coordinating with the front-end and data analysis components (e.g., AWS EMR, Google Dataproc, Azure HDInsight, Spark, etc.). It retrieves the results (follower and followee counts) and serves them to the front-end via APIs.

You need to provide a short description (max 100 words) of the set-up of your back-end. For example, about the technologies used and how you use the data analysis component.

3. Data Analysis Component (2 marks)

This component uses cloud-based services (e.g., AWS EMR, Google Dataproc, Azure HDInsight, etc.), which support Hadoop and frameworks like Spark, for processing the Twitter dataset in parallel, calculating follower and followee counts. This will contain the data analysis functions that perform the bulk of the necessary tasks.

You configure it by selecting a cloud service, specifying the dataset location in cloud storage (e.g., AWS S3, Google Cloud Storage, Azure Blob Storage, etc.), and uploading the necessary scripts to process the input line-by-line using Hadoop or similar frameworks (e.g., Spark, etc.). The cloud provider manages the distribution across nodes, showcasing scalable computing.

In your submission, please include the necessary source code for the data analysis functions as well as a few screenshots to show that you actually ran your solution *on the cloud*.